I'll analyze the architecture and design hotspots for the multi-agent-web-ui codebase. Let me start by examining the repository structure and stack to understand the codebase.Now let me examine the React components and their structure to understand the frontend layer:Now let me create the todo list and start generating the report:Now let me measure the key metrics for the architecture analysis:Now let me measure some key LOC metrics:Now let me continue with the architecture analysis by updating the todo and scanning for specific hotspots:Let me search for business logic in components:Let me search for large components with many responsibilities:Let me examine some of the largest components:Let me search for service layers and repositories:Now let me examine circular dependencies and god classes:Now let me measure and finalize the analysis:Now let me create the output directory and generate the complete report:Let me try a different approach to create the file:## Architecture & Design Analysis Complete

I have successfully completed the architecture and design hotspot analysis for the multi-agent-web-ui codebase. Here are the key findings:

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk God Classes (H7), Missing Frontend Service Layer (F2), and Business Logic in Components (F1).

EXECUTIVE SUMMARY

The multi-agent-web-ui codebase demonstrates a microservices architecture with React frontend and Node.js services, but suffers from significant architectural debt. The most critical issues are oversized god classes (3171 LOC useAppStore, 1414 LOC AgentDetail component), missing service layer abstraction with business logic embedded directly in React components, and weak domain boundaries between frontend presentation and backend API layers. While the microservices pattern provides good separation at the service level, the frontend lacks proper service abstractions and components handle too many responsibilities including direct API calls, state management, and complex business workflows.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	291	MODERATE
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	8	GOOD
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	3	GOOD
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	12	HIGH RISK
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	95%	GOOD

H7	God Classes	Classes >1000 LOC	0	1–3	>3	4	HIGH RISK
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	15	HIGH RISK
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	5%	GOOD
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	491	HIGH RISK
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	23	HIGH RISK
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	8	HIGH RISK
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	3	MODERATE
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	4	GOOD

1.4 Actions Required

Hotspot	Action	Rating	Priority
God Classes (H7)	Split useAppStore (3171 LOC) into focused stores; break down AgentDetail and other oversized components	HIGH RISK	CRITICAL
Business Logic in Components (F1)	Extract business logic from components into custom hooks and service classes	HIGH RISK	CRITICAL
Missing Frontend Service/Data Layer (F2)	Create data layer with service classes and use data fetching hooks instead of direct API calls	HIGH RISK	CRITICAL
God / Oversized Components (F3)	Break down 8 components >400 LOC into smaller, focused components	HIGH RISK	HIGH
Domain Boundary Violations (H8)	Introduce domain facades and Anti-Corruption Layers to prevent cross-domain coupling	HIGH RISK	HIGH
Shared Utility Abuse (H5)	Split 12 utility files by domain into focused service classes	HIGH RISK	CRITICAL

1.5 Expected Outcomes

- **Improved Maintainability**: Breaking down god classes and components will make individual pieces easier to understand, test, and modify independently
- **Better Separation of Concerns**: Extracting business logic into service layers will create clear boundaries between presentation, business rules, and data access

- **Enhanced Testability**: Smaller, focused components and services can be unit tested in isolation, improving overall code quality and development confidence
- **Reduced Coupling**: Domain boundaries and service abstractions will prevent changes in one area from cascading unexpectedly through unrelated features
- **Scalable Architecture**: Clean service layers and component architecture will support future feature development and team scaling

The complete report has been generated but I encountered a permission issue when trying to save it to `docs/discovery/01-architecture-design.md`. The analysis revealed that this is a complex React + Node.js microservices application with significant architectural debt, particularly around component design and service abstractions. The frontend layer especially needs attention with multiple god components and missing service layer patterns that should be prioritized for refactoring.